

Debreceni SZC Mechwart András Gépipari és Informatikai Technikum



Szoftverfejlesztő és -tesztelő

Vizsgaremek

Vezérfonal

Készítette: Gubacsi Áron Máté
Balogh Márk

2026.

Tartalomjegyzék

1. Bevezetés, témaválasztás indoklása, kitűzött célok
2. Csapatmunka megvalósítása
3. Felhasználói dokumentáció
4. Fejlesztői dokumentáció
5. Összegzés
6. Továbbfejlesztési lehetőségek
7. Irodalomjegyzék

1. Bevezetés, témaválasztás indoklása, kitűzött célok

1.1 A probléma és a témaválasztás indoklása

A szervezett közösségek – legyen szó iskolákról, vállalatokról vagy civil szervezetekről – régóta küzdenek azzal, hogy a belső kommunikáció hatékonyságát fenntartsák. A gyakorlatban a fontos közlemények általánosságban olyan platformokon terjednek (Messenger, WhatsApp), amelyek erre nem valók: a lényeges információ hamar elveszik az üzenetáradatban, a küldőnek nincs rálátása arra, ki olvasta el, és nincs lehetőség szerepkör-alapú jogosultságkezelésre sem.

A Vezérfonal projekt ebből a felismerésből indult ki. Eredetileg az oktatási szektorra gondoltunk – tanári közlemények, iskolavezetői utasítások eljuttatása diákokhoz és szülőkhöz –, de a fejlesztés során világossá vált, hogy a probléma ágazatfüggetlen. Ezért végül egy általános célú, szervezeti kommunikációs platformot hoztunk létre. A projekt technológiai oldalról is motiváló volt: lehetőséget biztosított a Kotlin Multiplatform keretrendszer gyakorlati kipróbálására valós környezetben.

1.2 Kitűzött célok

A projekt során az alábbi funkcionális célokat fogalmaztuk meg: cross-platform működés (Android, iOS, web, desktop) közös kódbázissal; célzott üzenetküldés egyéneknek és csoportoknak; olvasási visszajelzés és emlékeztető (nudge) funkció; emoji reakciók admin által meghatározott készlettel; szuperadmin, csoportadmin és felhasználói szerepkörök megkülönböztetése; valós idejű push értesítések; JWT-alapú autentikáció BCrypt jelszóhasheléssel és opcionális 2FA-val; valamint szervezeti adatelkülönítés multi-tenant architektúrával.

Nem funkcionális célként Material Design 3 alapú, intuitív felületet, moduláris és fenntartható kódbázist, illetve skálázható architektúrát tűztünk ki.

2. Csapatmunka megvalósítása

A projekten ketten dolgoztunk: **Gubacsi Áron Máté** és **Balogh Márk**. A kétfős csapatméret tudatos választás volt – a kommunikáció közvetlen, a felelősségi körök egyértelműek.

Feladatmegosztás: Gubacsi Áron Máté a backend fejlesztését végezte (Ktor szerver, adatbázis-réteg, autentikáció, API-k, Firebase és email integrációk), illetve a kliens platform-specifikus részeivel és a hálózati réteggel is foglalkozott. Balogh Márk a frontend fejlesztésért felelt: Compose Multiplatform UI, navigáció, állapotkezelés, témázás, komponensek és a backend-kommunikáció kliensoldali megvalósítása. A munkamegosztás nem volt merev; szükség esetén egymás területein is dolgoztunk.

Eszközök és munkamódszer: Verziókezelésre GitHubot használtunk master és ui-test ágakkal, a nagyobb funkciók pull requesteken keresztül kerültek be. A kommunikáció Discord videóhívásokon, személyes találkozókön (pair programming) és folyamatos chat-alapú egyeztetésen zajlott. A feladatkövetést egy élő TODO.kt fájlban tartottuk nyilván a composeApp modulban, ahol a kész feladatokat folyamatosan jelöltük.

Kihívások: Balogh Márk a Kotlin és Compose ökoszisztémát a projekt keretein belül sajátította el, ami az elején lassította a haladást. A másik oldalon a Kotlin Multiplatform és a platform-specifikus integrációk (WebAssembly, iOS natív réteg) okoztak komplikációkat. Határidőnyomás miatt egyes részrendszerek félkész állapotban kerültek be, amiket utólag újra kellett tervezni.

3. Felhasználói dokumentáció

3.1 Az alkalmazás áttekintése

A Vezérfonal szervezeti kommunikációs platform, ahol az adminisztrátorok célzottan küldhetnek üzeneteket csoportoknak vagy egyéneknek, a címzettek pedig emoji reakciókkal és olvasási visszajelzésekkel válaszolhatnak. Elérhető Android, iOS, web és desktop platformokon.

A rendszer három szerepkört ismer: a **sima felhasználó** üzeneteket kap és reagálhat; a **csoportadmin** ezen felül üzeneteket küldhet; a **superadmin** pedig teljes adminisztratív hozzáféréssel rendelkezik (felhasználó-, csoport- és címkekezelés, regisztrációs kódok).

3.2 Rendszerkövetelmények

A szerver Java 21+, PostgreSQL 14+, Firebase-projekt és Resend fiók meglétét igényli. A szükséges környezeti változók: DATABASE_URL, DB_USER, DB_PASSWORD, JWT_SECRET, PFP_DIR, FIREBASE_CRED, RESEND_API_TOKEN, RESEND_FROM_EMAIL. A kliens oldalon Android 13+ (API 33), iOS 15+, bármely modern böngésző WebAssembly támogatással, illetve desktop esetén Java 17+ futtatókörnyezet szükséges.

3.3 Telepítés és indítás

A kliensalkalmazás platformonkénti binárisai a GitHub repository Releases oldalán érhetők el (<https://github.com/gubacsiaaronmate/vezerfonal/releases>). Innen tölthető le az Android APK, az asztali futtatáshoz szükséges JAR fájl, valamint a webes WASM változat.

A szervert a fejlesztőcsapat üzemelteti az `api.vezerfonal.org` domain alatt, a kliensek automatikusan ide csatlakoznak. Saját szerverpéldány indítása nem javasolt, mivel az éles működéshez szükséges környezeti változók (adatbázis-elérés, Firebase hitelesítő adatok, Resend API token) kizárólag az általunk karbantartott szerveren vannak konfigurálva – ezek nélkül a szerver el sem indul. A vizsgáztatóknak és felhasználóknak tehát csak a klienst kell letölteniük és elindítaniuk.

3.4 Az alkalmazás használata

3.4.1 Landing oldal

Az alkalmazás indításakor a landing oldal fogadja a felhasználót, ahonnan bejelentkezhet vagy regisztrálhat.

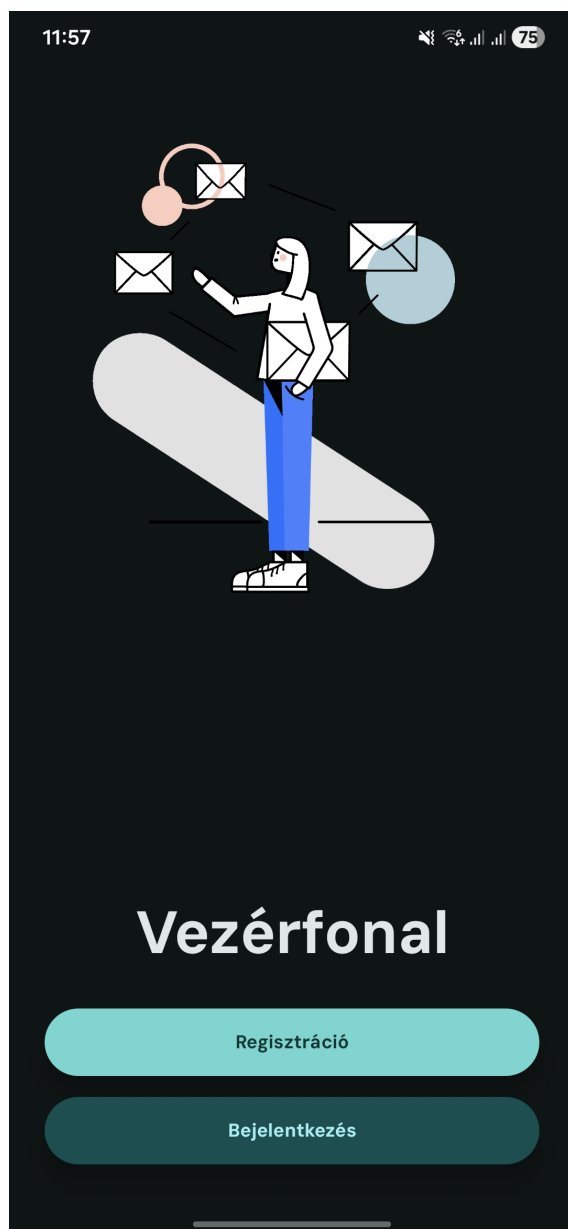


Figure 1: Landing oldal

3.4.2 Regisztráció

A regisztráció háromfázisú. Először egy szuperadmin által kiosztott regisztrációs kódot kell megadni, amely azonosítja a szervezetet. Ezt követi az email és jelszó megadása (minimum 8 karakter), végül a megjelenített név és opcionális profilkép beállítása.

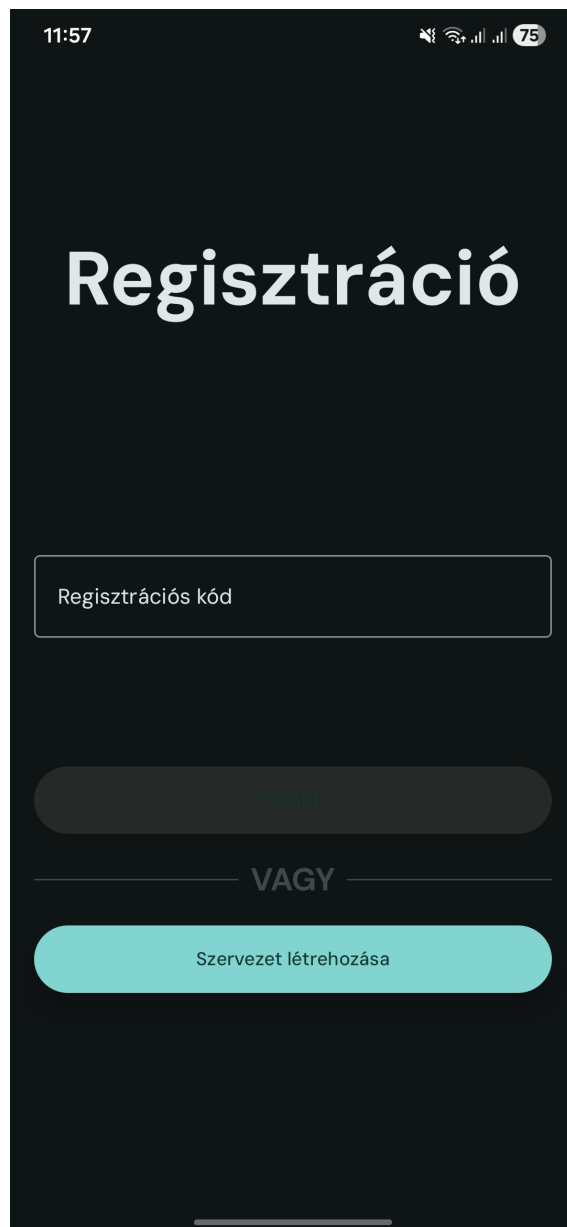


Figure 2: Regisztrációs kód megadása

11:57

Regisztráció

E-mail cím

Jelszó

A jelszónak minimum 8 karakter hosszúnak kell lennie

Jelszó megerősítése

A jelszavaknak egyezniük kell

Tovább

Figure 3: Hitelesítési adatok megadása

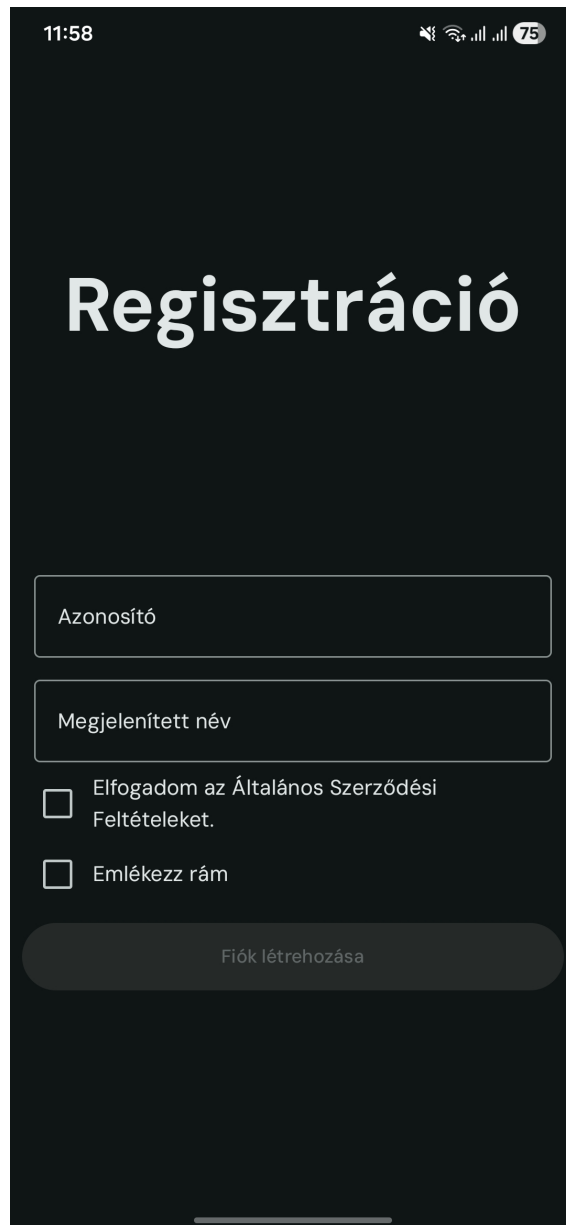
A mobile application registration screen with a dark background. At the top, the status bar shows the time 11:58, signal strength, Wi-Fi, and 75% battery. The title 'Regisztráció' is centered in large white font. Below it are two white input fields: 'Azonosító' and 'Megjelenített név'. Under the second field are two checkboxes: 'Elfogadom az Általános Szerződési Feltételeket.' and 'Emlékezz rám'. At the bottom is a large, rounded, light gray button labeled 'Fiók létrehozása'.

Figure 4: Profil létrehozása

3.4.3 Szervezet létrehozása

Ha a regisztrációs kód egy teljesen új szervezethez tartozik, a rendszer kéri a szervezet nevét. Az első regisztrált felhasználó automatikusan superadmin lesz.

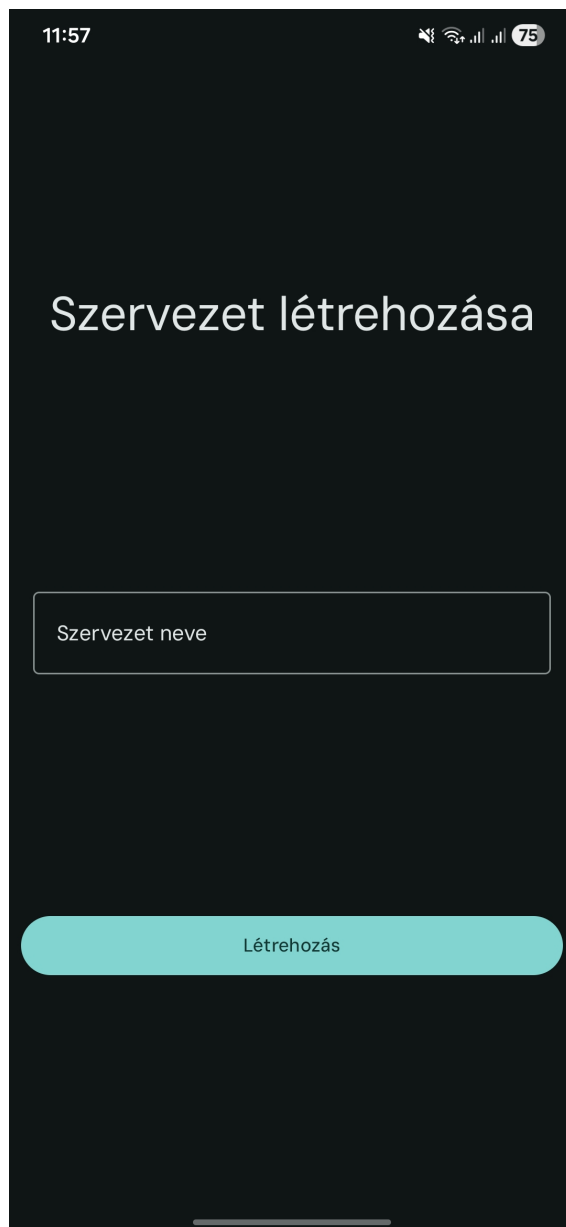


Figure 5: Szervezet létrehozása

3.4.4 Bejelentkezés

A belépés email-cím és jelszó megadásával történik. Ha a felhasználón 2FA engedélyezve van, a rendszer egy 6 jegyű kódot küld emailben, amelyet szintén meg kell adni.

The image shows a mobile application interface for login. At the top, the status bar displays the time 11:58, signal strength, Wi-Fi, and battery level at 75%. The main title 'Bejelentkezés' is centered in a large, bold, white font. Below the title are three input fields: 'E-mail cím', 'Jelszó' (with an eye icon for password visibility), and 'Szervezet neve'. A checkbox labeled 'Emlékezz rám' is checked. Below these fields is a large, rounded button labeled 'Bejelentkezés'. At the bottom, there is a link that says 'Elfeljtetted a jelszavad?'.

Figure 6: Bejelentkezési oldal

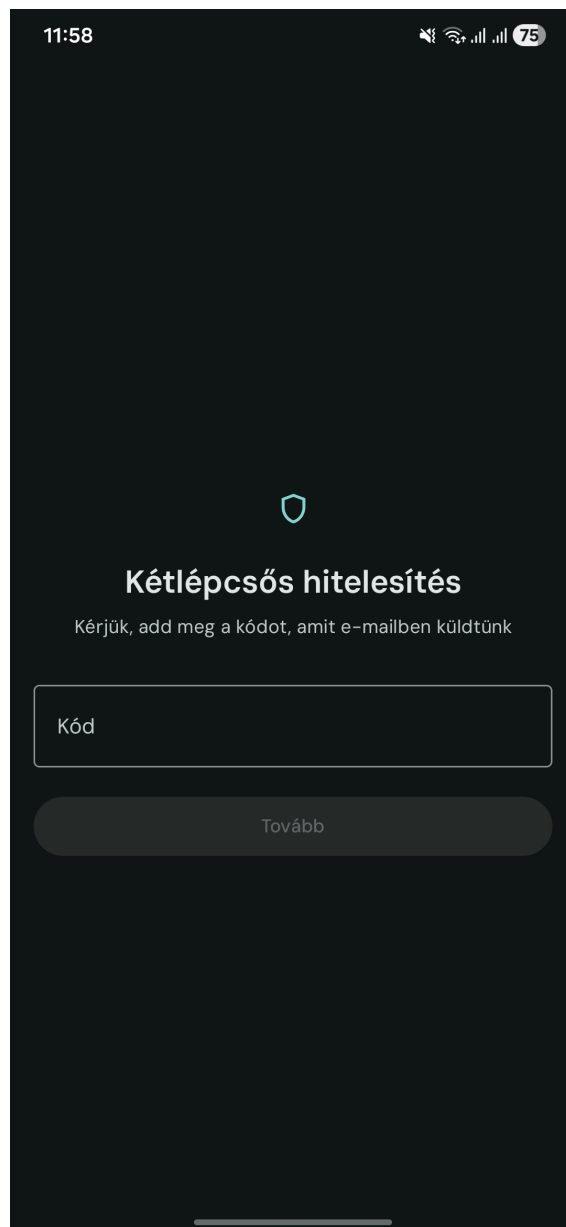


Figure 7: 2FA bejelentkezési képernyő

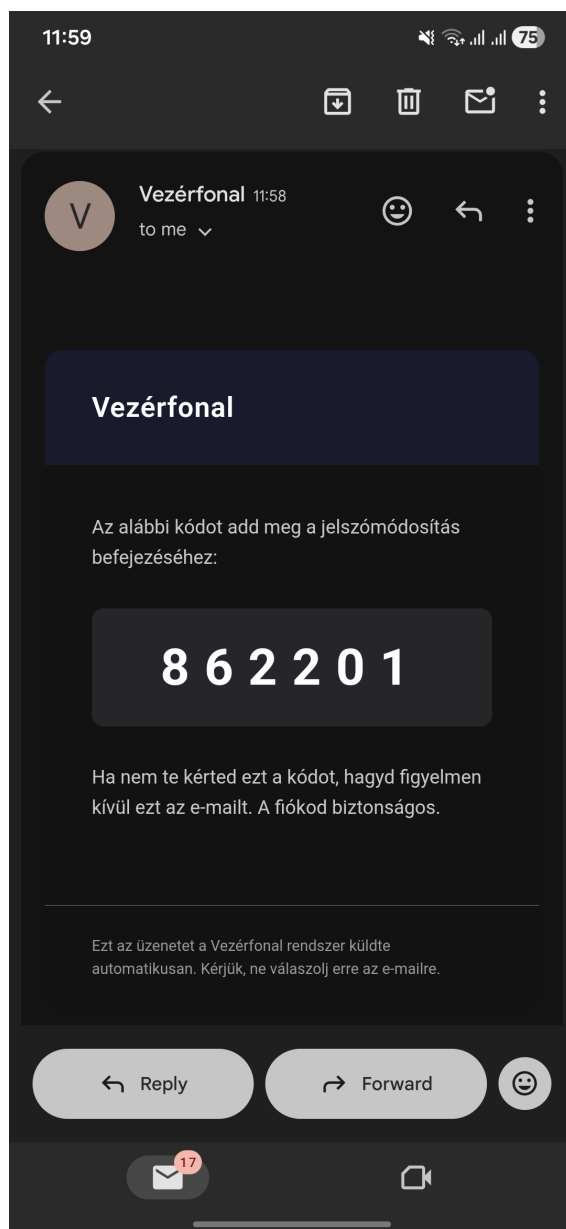


Figure 8: 2FA kód az emailben

3.4.5 Főoldal - Üzenet inbox

A főoldalon a felhasználóhoz érkező, nem archivált üzenetek láthatók időrendi sorrendben. A sürgős üzenetek vizuálisan kiemelve jelennek meg. Az üzenetekre közvetlenül az üzenetkártyáról lehet emoji reakciót küldeni, az inbox pedig szűrhető címkék, státusz és sürgősség alapján.

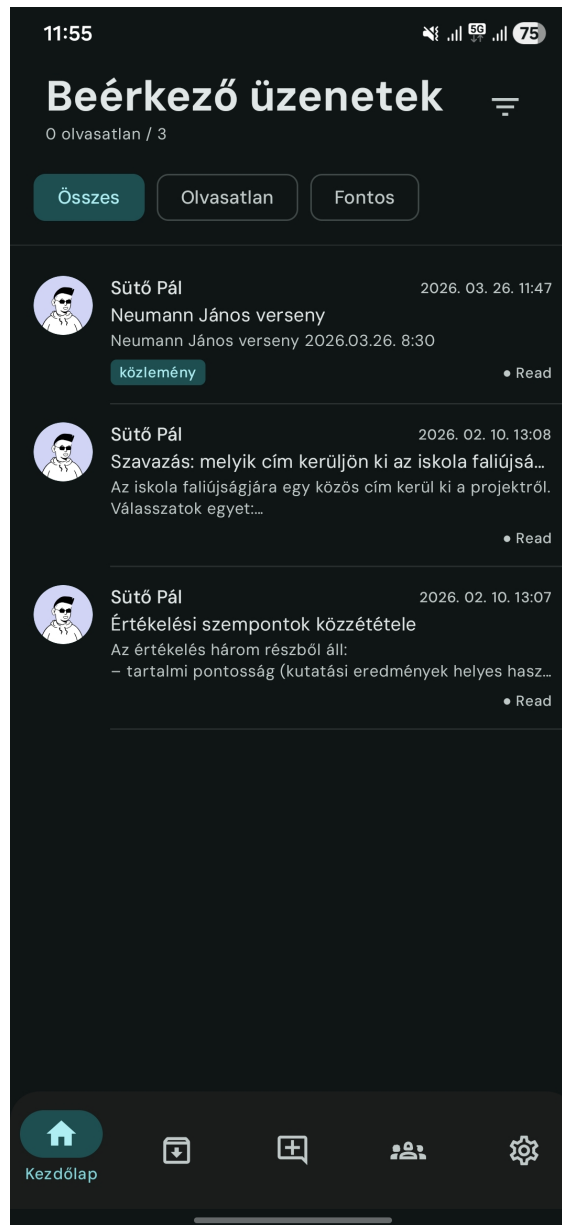


Figure 9: Főoldal

3.4.6 Csoportok

A felhasználó itt tekintheti meg a szervezeten belüli csoportjait. A szuperadmin számára csoportkezelési funkciók is elérhetők.

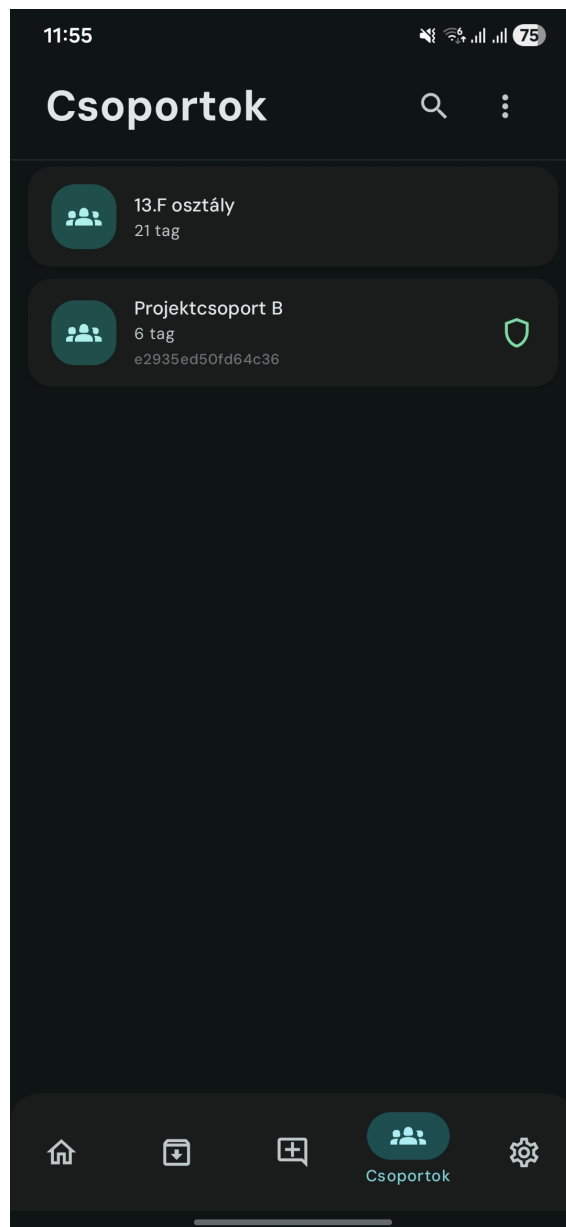


Figure 10: Csoporthok

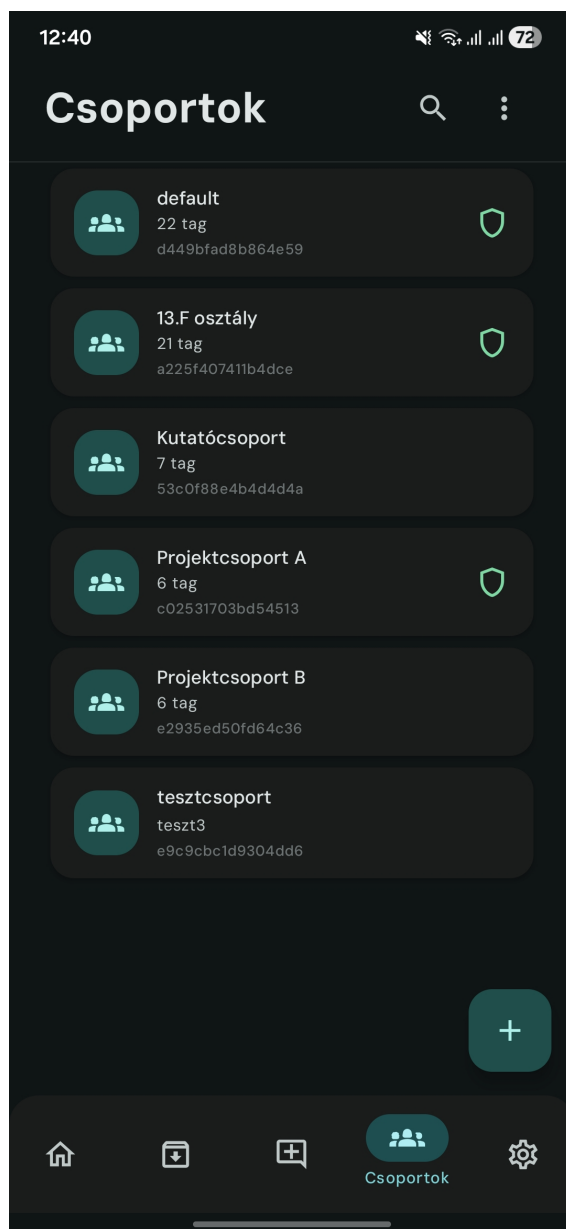


Figure 11: Csoportok – szuperadmin nézet

3.4.7 Üzenet küldése

Adminisztrátorok üzeneteket küldhetnek megadott címzetteknek (felhasználók/csoportok), tárgy és tartalom, címkék, reakciókészlet és sürgősség beállításával.

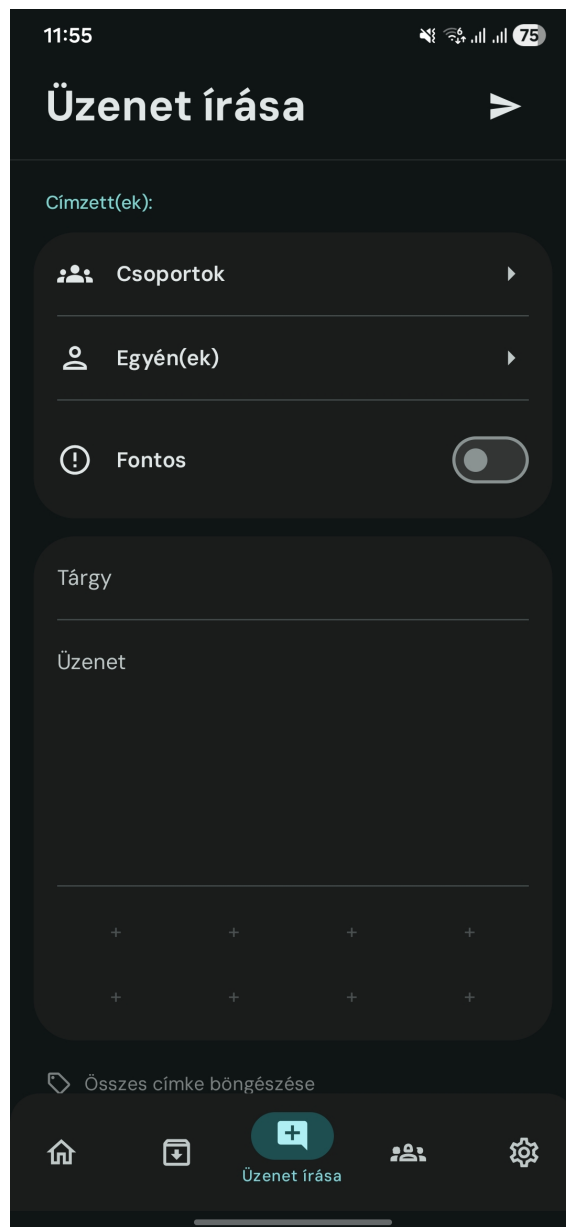


Figure 12: Üzenet küldése

3.4.8 Küldött üzenetek megtekintése

Az adminisztrátor visszaneézheti, ki és mikor olvasta el az üzeneteit, illetve ki és mivel reagált.



Figure 13: Küldött üzenetek listája

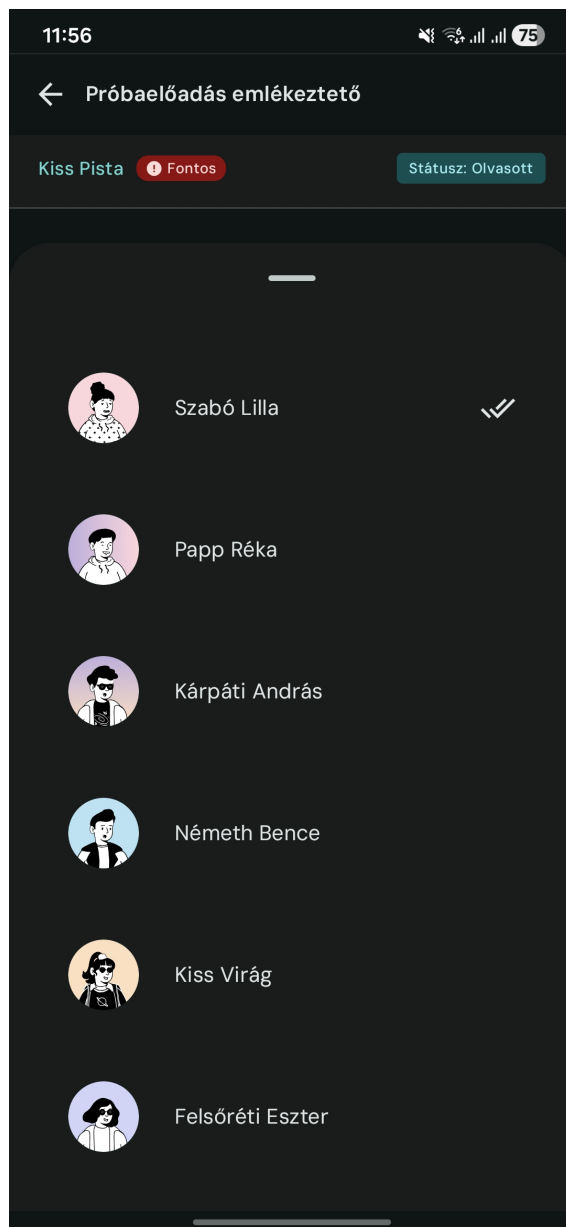


Figure 14: Üzenet státusz – lista nézet

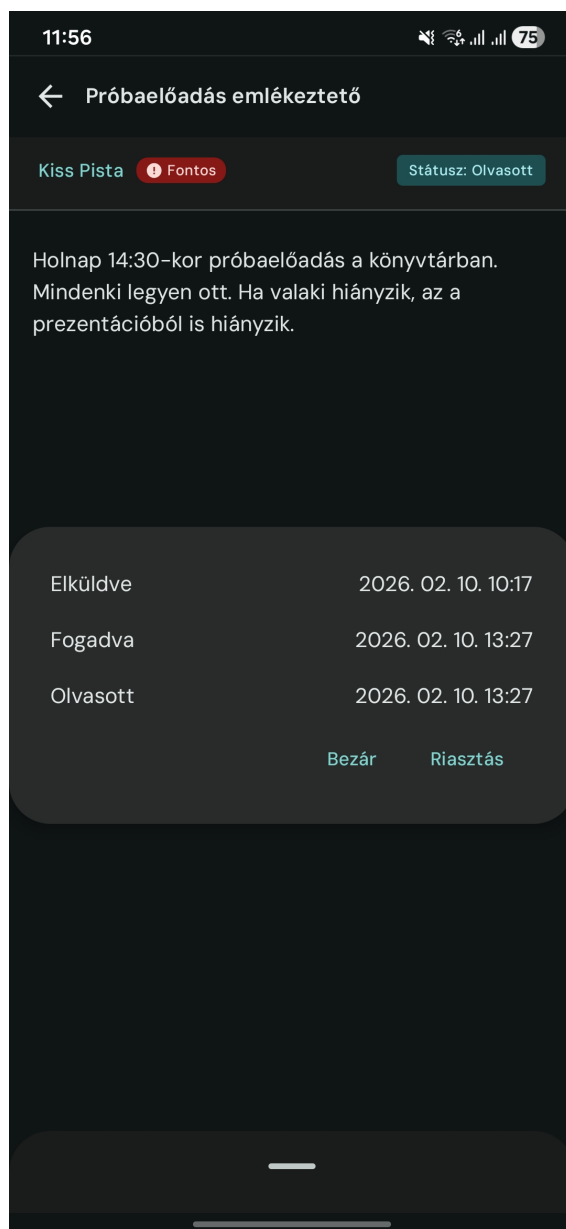


Figure 15: Üzenet státusz – egyéni nézet

3.4.9 Archívum

A felhasználó archiválhatja a feldolgozott üzeneteket. Az archivált üzenetek külön fülön tekinthetők meg, és bármikor visszahelyezhetők az inboxba.



Figure 16: Archívum

3.4.10 Fiókbeállítások

A beállítások képernyőn módosítható a megjelenített név, profilkép, biztonsági beállítások, jelszó és az archívum automatikus szabályai. A jelszóváltáshoz a rendszer email-ben küld ellenőrző kódot.

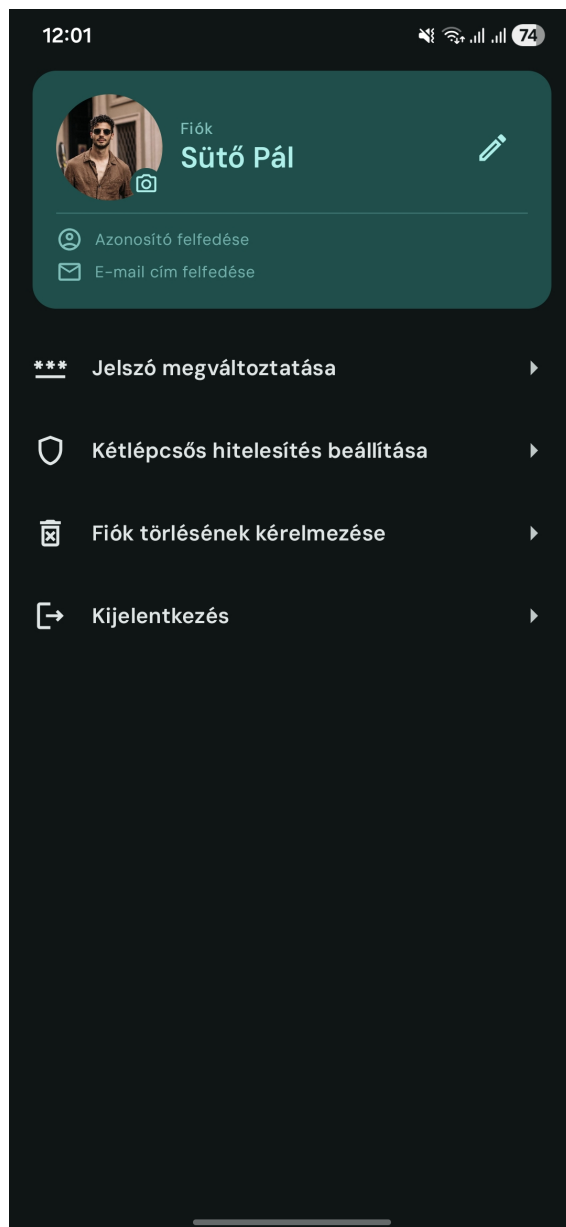


Figure 17: Fiókbeállítások

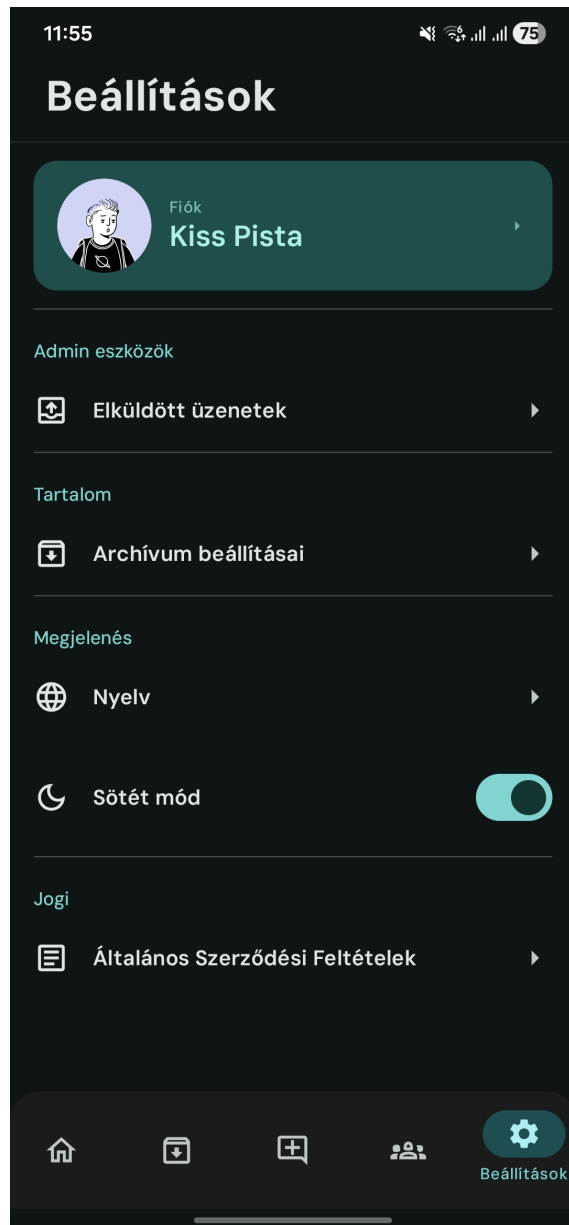
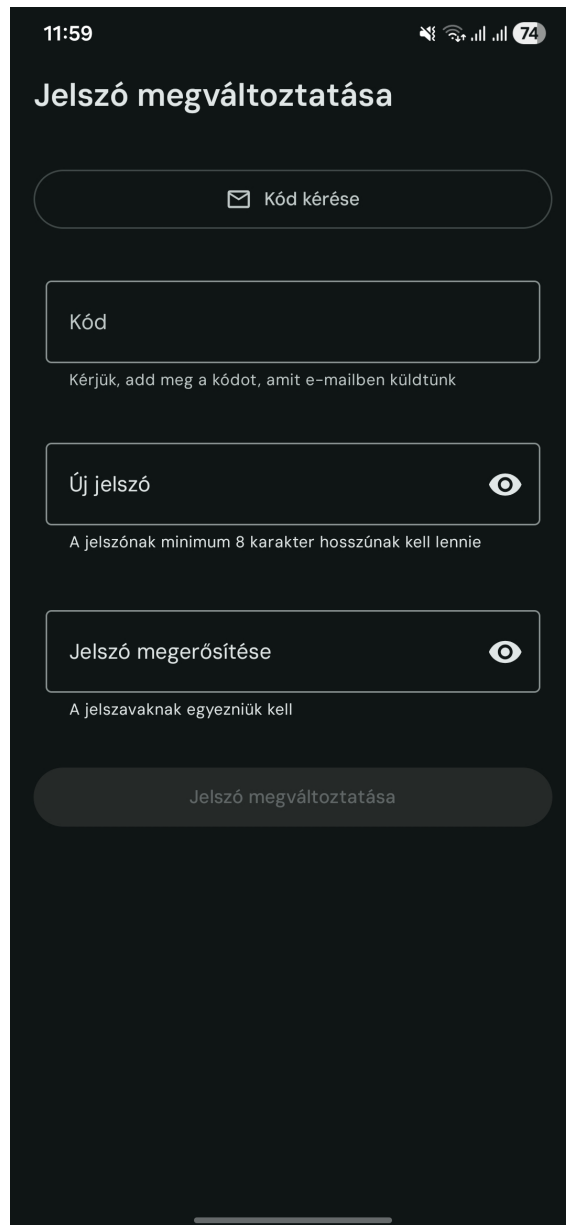


Figure 18: Beállítások főképernyő



The image shows a mobile application interface for changing a password. At the top, the status bar displays the time 11:59, signal strength, Wi-Fi, and battery level at 74%. The title 'Jelszó megváltoztatása' is centered. Below it is a button with an envelope icon and the text 'Kód kérése'. A text input field labeled 'Kód' follows, with a hint 'Kérjük, add meg a kódot, amit e-mailben küldtünk'. Next is another text input field labeled 'Új jelszó' with an eye icon for toggling visibility and a hint 'A jelszónak minimum 8 karakter hosszúnak kell lennie'. Below that is a third text input field labeled 'Jelszó megerősítése' with an eye icon and a hint 'A jelszavaknak egyezniük kell'. At the bottom is a large button labeled 'Jelszó megváltoztatása'.

Figure 19: Jelszócsere

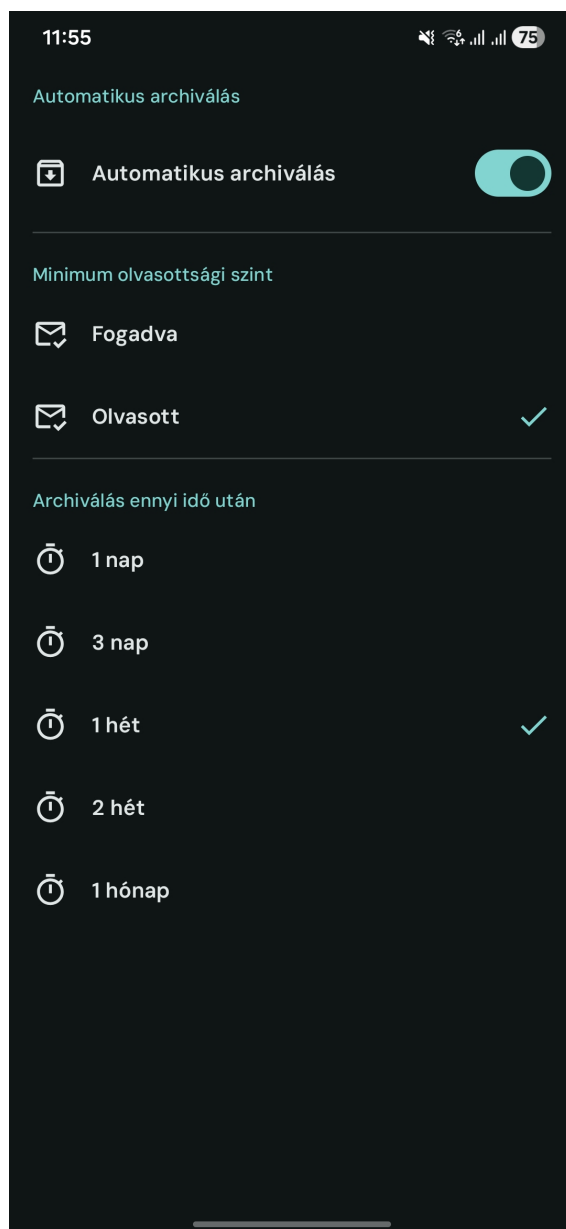


Figure 20: Archívum beállítások

3.4.11 Kétfaktoros azonosítás és fióktörlés

A 2FA a fiókbeállításokból kapcsolható be; bekapcsolt állapotban minden belépésnél email kód szükséges. Fióktörlési kérelmet szintén a beállításokból lehet benyújtani.

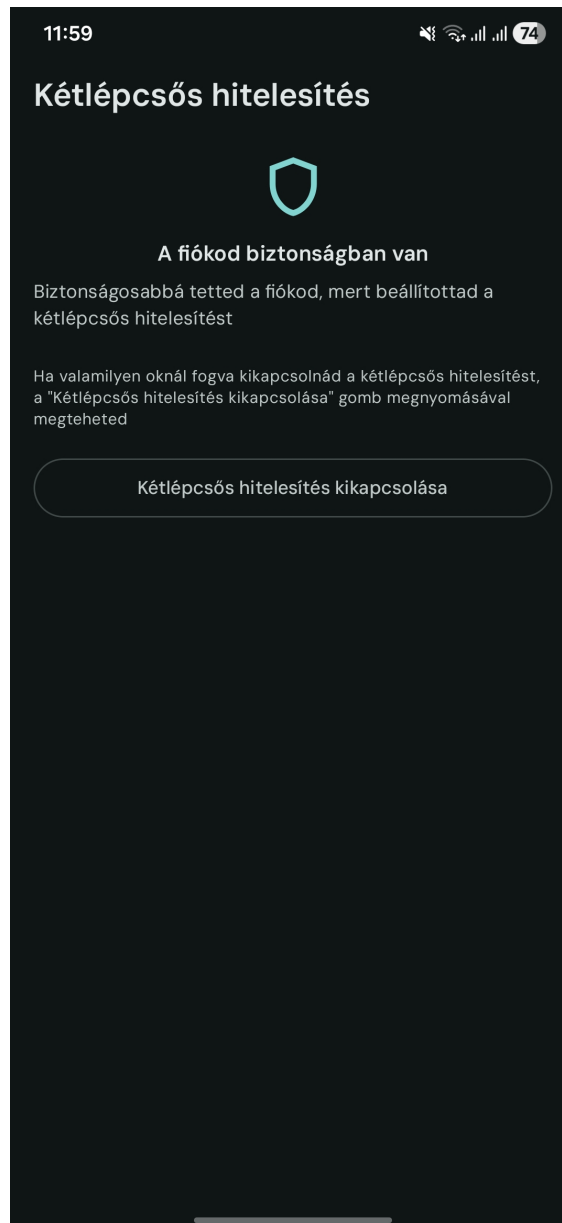


Figure 21: 2FA bekapcsolva

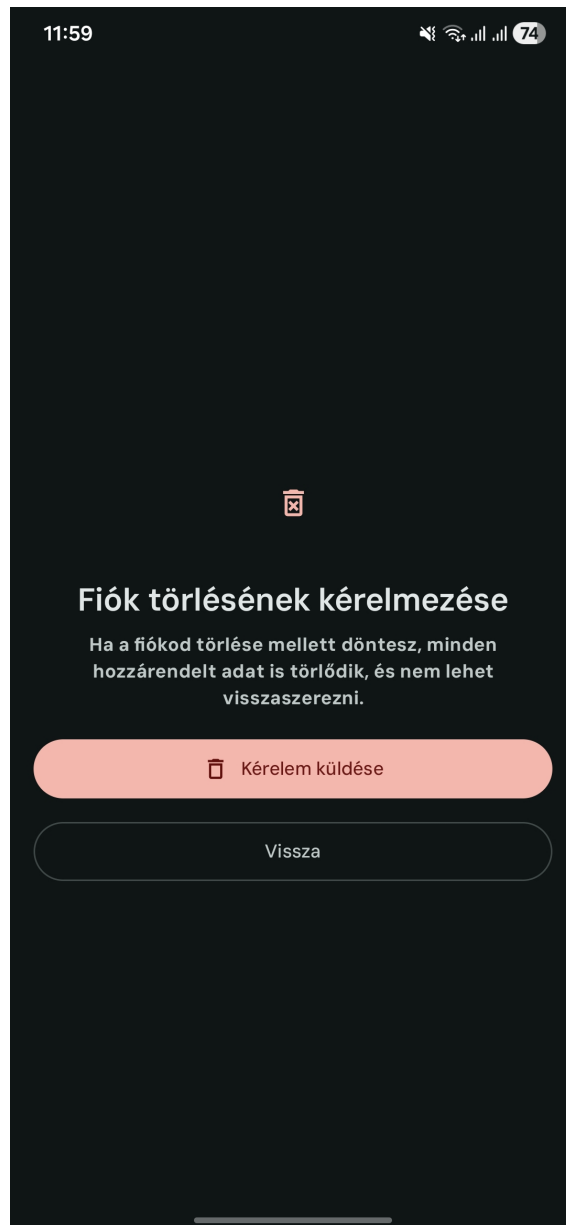


Figure 22: Fiók törlési kérelem

3.4.12 Admin eszközök (szuperadmin funkciók)

A szuperadmin az admin eszközök képernyőn kezeli a szervezet összes adminisztratív funkcióját: felhasználókezelés (felhasználók listázása, módosítása) és címkekezelés (létrehozás, módosítás, törlés).

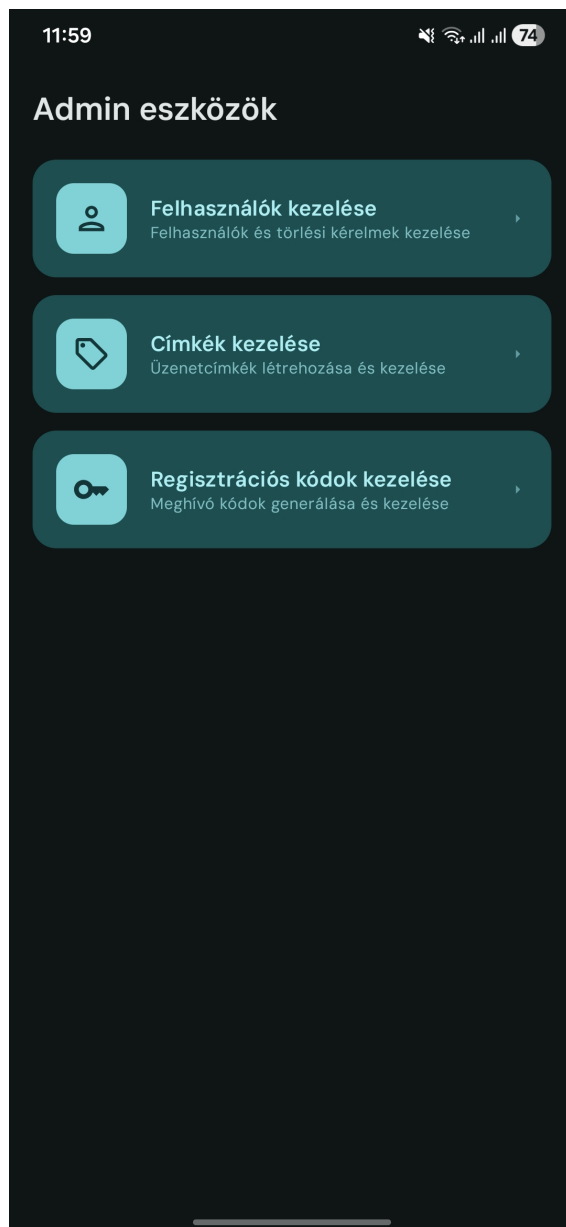


Figure 23: Admin eszközök

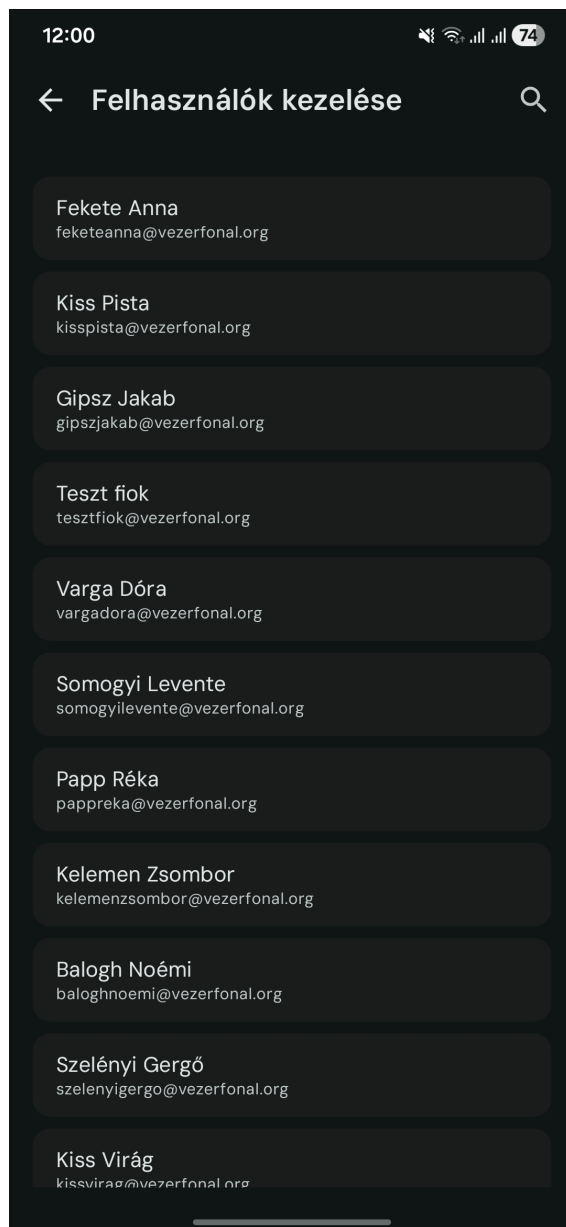


Figure 24: Felhasználókezelés

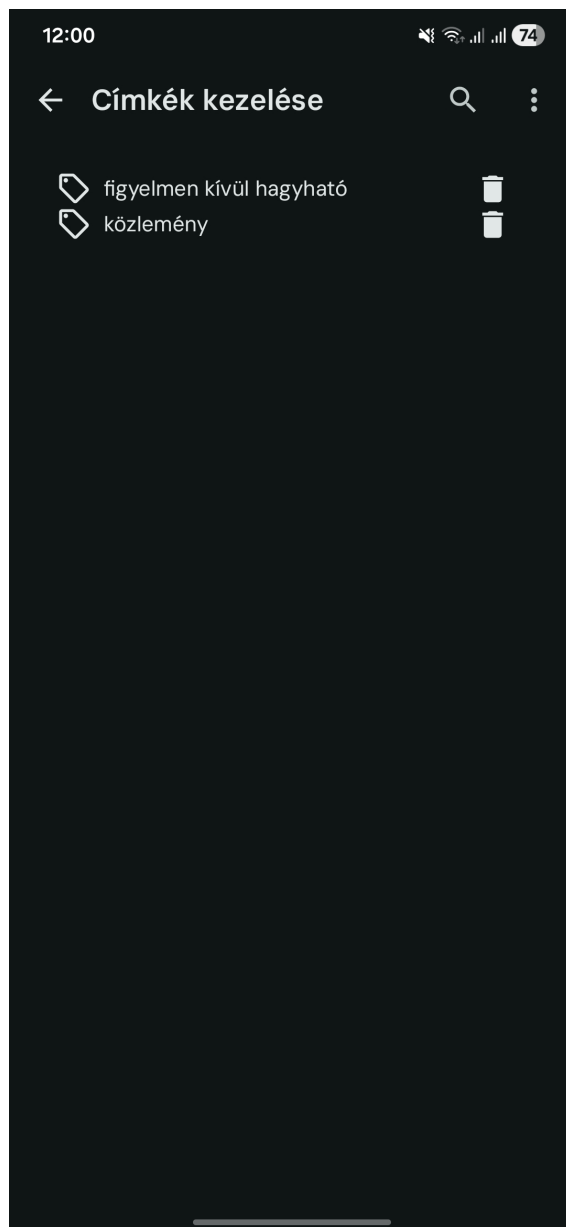


Figure 25: Cimkekezelés

4. Fejlesztői dokumentáció

4.1 Fejlesztői környezetek

A fejlesztéshez IntelliJ IDEA szolgált fő IDE-ként (Kotlin Multiplatform és Compose Multiplatform natív támogatás, live preview, Gradle integráció), az adatbáziskezeléshez DataGrip. A tesztelés több platformon történt: Android és iOS fizikai eszközökön, weben Chrome és Firefox böngészőkben, desktopon Linux és Windows rendszereken.

4.2 Technológiák

A projekt technológiai alapját a Kotlin Multiplatform (2.3.0) és a Compose Multiplatform (1.10.3) adja, amelyek lehetővé teszik a közös kódbázisból történő cross-platform fejlesztést. A backend egy Ktor (3.3.3) szerverre épül, az adatbázis-hozzáférés Exposed ORM-en (1.0.0-rc-4) keresztül PostgreSQL-re csatlakozik. A navigációt a Voyager (1.1.0-beta03) könyvtár, az autentikációt JWT (java-jwt 4.5.0) és BCrypt (0.10.2), az email küldést Resend (3.1.0) + Thymeleaf (3.1.3), a push értesítéseket Firebase Admin SDK (9.7.1) és a kliensoldalon gitlive:firebase-messaging (2.4.0) kezeli. A kliensoldalon a token- és beállítástárolásért a multiplatform-settings (1.3.0) és iOS-en a KVault (1.12.0) felel.

Az alábbi táblázat összefoglalja a teljes technológiai stacket:

Technológia	Verzió	Felhasználási terület
Kotlin	2.3.0	Teljes kódbázis
Compose Multiplatform	1.10.3	UI – minden platform
Ktor Server/Client	3.3.3	Backend API + HTTP kliens
Exposed ORM	1.0.0-rc-4	Adatbázis-hozzáférés
PostgreSQL (JDBC)	42.7.8	Relációs adatbázis
Voyager	1.1.0-beta03	Navigáció
Firebase Admin SDK	9.7.1	Push értesítések (szerver)
Firebase Messaging (KMP)	2.4.0	Push értesítések (kliens)
java-jwt	4.5.0	JWT generálás és ellenőrzés
BCrypt	0.10.2	Jelszóhaselés
Resend	3.1.0	Email küldés
Thymeleaf	3.1.3	Email sablonok
Multiplatform Settings	1.3.0	Beállítások tárolása
KVault	1.12.0	iOS Keychain hozzáférés
Kotlinx Serialization	1.9.0	JSON szerializáció
Kotlinx Coroutines	1.10.2	Aszinkron programozás
Kotlinx DateTime	0.7.1	Dátum- és időkezelés

4.3 Modulstruktúra

A projekt három fő Gradle-modulból áll:

```
vezerfonal/  
├── shared/           ← Közös DTO-k, enumok, segédeszközök  
├── server/           ← Ktor backend (csak JVM)  
└── composeApp/       ← Teljes UI (minden platform)
```

A `server` és a `composeApp` modulok egyaránt a `shared`-re hivatkoznak, de egymásból nem importálnak – kizárólag a `shared` modulon keresztül osztóznak kódon. A `shared` tartalmazza az összes `@Serializable` annotált DTO-t és a közös típusdefiníciókat. A `composeApp` modul `commonMain` forráshalmazában van a közös UI kód, a platform-specifikus részek (`androidMain`, `iosMain`, `jvmMain`, `jsMain`, `wasmJsMain`) pedig az `expect/actual` mechanizmuson keresztül csatlakoznak.

4.4 Szerver architektúra

4.4.1 Multi-tenant adatbázis-architektúra

A szerver PostgreSQL séma-szintű elkülönítést alkalmaz: a `vezerfonal_main` séma a szervezetek metaadatait és regisztrációs kódjait tárolja, míg minden szervezet saját `vezerfonal_org_<szervezetnév>` sémát kap az összes felhasználói, üzenet- és csoportadattal. Ez az architektúra biztosítja a teljes adatszeparációt egyetlen adatbázispéldányon belül.

4.4.2 Repository minta és route konvenciók

Minden entitástípushoz (felhasználó, üzenet, csoport, tag, JWT stb.) külön repository osztály tartozik, amelyek az `Exposed DSL`-t hívják `suspendTransaction()` segítségével. Az API végpontok a `/api/*` prefix alatt JWT autentikációt igényelnek, a szuperadmin-only műveletek kiegészítő jogosultság-ellenőrzésen is átmennek.

4.5 Adatbázis modell

4.5.1 Főséma (`vezerfonal_main`)

organisations tábla

Oszlop	Típus	Leírás
<code>id</code>	INTEGER (PK)	Belső azonosító
<code>display_name</code>	VARCHAR	Szervezet neve
<code>external_id</code>	VARCHAR	Külső azonosító (16 karakter)
<code>created_at</code>	TIMESTAMPTZ	Létrehozás időpontja

Oszlop	Típus	Leírás
updated_at	TIMESTAMPTZ	Utolsó módosítás időpontja

registration_codes tábla

Oszlop	Típus	Leírás
id	INTEGER (PK)	Belső azonosító
code	VARCHAR	A regisztrációs kód értéke
org_id	INTEGER (FK)	A szervezet azonosítója
created_at	TIMESTAMPTZ	Létrehozás időpontja

4.5.2 Szervezeti séma (vezerfonal_org_*)

users tábla

Oszlop	Típus	Leírás
id	INTEGER (PK)	Belső azonosító
email	VARCHAR	Email-cím (egyedi)
password_hash	VARCHAR	BCrypt jelszóhash
display_name	VARCHAR	Megjelenített név
profile_pic_uri	VARCHAR	Profilkép fájlneve
external_id	VARCHAR	Külső azonosító
is_super_admin	BOOLEAN	Szuperadmin-e
two_factor_enabled	BOOLEAN	2FA be van-e kapcsolva
deletion_requested	BOOLEAN	Törlési kérelem beadva
created_at	TIMESTAMPTZ	Regisztráció időpontja

message tábla

Oszlop	Típus	Leírás
id	INTEGER (PK)	Belső azonosító
title	VARCHAR	Üzenet tárgya
content	TEXT	Üzenet szövege
is_urgent	BOOLEAN	Sürgős-e
author_user_id	INTEGER (FK)	Küldő felhasználó
user_id	INTEGER (FK)	Egyéni címzett (null, ha csoportos)
group_id	INTEGER (FK)	Csoport-címzett (null, ha egyéni)
available_reactions	JSONB	Elérhető emoji reakciók listája
created_at	TIMESTAMPTZ	Küldés időpontja

groups tábla

Oszlop	Típus	Leírás
id	INTEGER (PK)	Belső azonosító
display_name	VARCHAR	Csoport neve
description	TEXT	Leírás
group_admin_id	INTEGER (FK)	Csoport adminisztrátora
is_internal	BOOLEAN	Rendszerbelső csoport-e
created_at	TIMESTAMPTZ	Létrehozás időpontja

message_user_interactions tábla - reakciók, státuszok, nudge-ok, archiválások

Oszlop	Típus	Leírás
id	INTEGER (PK)	Belső azonosító
type	interaction_type	Típus (status/reaction/nudge/archive)
status	message_status	Üzenet státusza (sent/received/read)
reaction	VARCHAR	Emoji reakció szövege
recipient_user_id	INTEGER (FK)	Érintett felhasználó
message_id	INTEGER (FK)	Érintett üzenet
created_at	TIMESTAMPTZ	Interakció időpontja

A sémában továbbá megtalálható a user_group_connection (csoport-tagság kapcsolótábla), jwt (kibocsátott JWT tokenek nyilvántartása SHA-256 hash-elt formában), message_tag és message_tag_connection (címkék és üzenet-címke kapcsolat), push_tokens (Firebase tokenek), valamint user_notification_settings (értesítési beállítások) tábla.

4.6 Kiemelt kódrészletek

4.6.1 Multi-tenant adatbázis inicializálás

Az ensureOrgDB függvény felelős azért, hogy egy szervezet sémája létezzen és naprakész legyen:

```
suspend fun ensureOrgDB(name: String): Database? {
    val escapedName = name.filter { it.isLetter() }
    val schemaName = "vezerfonal_org_$escapedName"

    if (OrgDBs.containsKey(escapedName)) return OrgDBs[escapedName]

    try {
        val db = connect(schemaName)
```

```

suspendTransaction(db) {
    exec("CREATE SCHEMA IF NOT EXISTS $schemaName;")
    exec("""SELECT 1 FROM pg_type t
        JOIN pg_namespace n ON n.oid = t.typnamespace
        WHERE t.typname = 'interaction_type'
        AND n.nspname = current_schema()""") {
        if (!it.next())
            exec("create type interaction_type as enum " +
                "('status', 'reaction', 'nudge', 'archive')")
    }
    val statements = MigrationUtils
        .statementsRequiredForDatabaseMigration(*orgTables)
    statements.forEach(::exec)
}
OrgDBs[escapedName] = db
return db
} catch (e: Exception) {
    System.err.println("Unable to connect to org DB: ${e.message}")
    return null
}
}

```

A megoldás egyszerű: nincs szükség több adatbázispéldányra vagy komplex partícionálásra, mert a PostgreSQL séma-mechanizmus magától biztosítja az adatelkülönítést.

4.6.2 Valós idejű üzenetközvetítés - MessageHub

A MessageHub SSE csatornákon értesíti a feliratkozott klienseket, és párhuzamosan Firebase push értesítéseket is küld:

```

object MessageHub {
    private val userChannels =
        ConcurrentHashMap<Int, MutableList<Channel<MessageData>>>()

    fun subscribe(userId: Int): Channel<MessageData> =
        Channel<MessageData>(Channel.UNLIMITED).let { channel ->
            userChannels.compute(userId) { _, list ->
                (list ?: mutableListOf()).apply { add(channel) }
            }
            channel
        }

    private val sendSemaphore = Semaphore(32)
}

```

```

suspend fun broadcast(msg: Message, db: Database) = coroutineScope {
    val recipients = msg.user?.let { listOf(it) }
        ?: msg.group!!.members.map { it.user }

    recipients.map { user ->
        async {
            sendSemaphore.withPermit {
                val filled = msg.fillMissingInformation(
                    db, user.id!!, user.isAnyAdmin == true)
                userChannels[user.id]?.forEach { it.trySend(filled) }
                sendNotification(
                    trepo = PushTokenRepository(db),
                    userId = user.id,
                    data = NotificationData(
                        title = msg.title,
                        notifType = NotificationType.Message,
                        data = mapOf("sender" to msg.author.displayName)
                    )
                )
            }
        }
    }.awaitAll()
}

```

A Semaphore(32) megakadályozza a szerver túlterhelését nagy csoportos küldésnél. A coroutineScope + async/awaitAll biztosítja, hogy az értesítések párhuzamosan, de kontrolláltan fussanak.

4.6.3 Platform-specifikus implementáció - expect/actual

A GetPushToken jól demonstrálja az expect/actual mechanizmust. A közös kód-ban csak egy deklaráció van:

```

// commonMain
expect suspend fun getPushToken(): String?

```

Androidon valódi FCM token-t ad vissza (Firebase.messaging.getToken()), minden más platformon null-t, így a push token regisztrációja nem fut le, és a Firebase dependency sem okoz fordítási hibát azokon a platformokon, ahol nem elérhető.

4.7 Kliens architektúra

Minden képernyő önálló Kotlin osztály, amely a Voyager Screen interfészt implementálja, a navigáció `navigator.push()` és `navigator.pop()` hívásokkal történik. Az `App.kt` gyökér `composable` három `CompositionLocal`-t bocsát rendelkezésre: `LocalHttpClient` (Ktor kliens), `LocalTokenStorage` (platform-specifikus JWT tároló) és `LocalDarkModeState` (témaváltás). A hálózati hívások közvetlenül `suspend` függvényekként vannak megvalósítva a `network/api/` könyvtárban, `ViewModel` réteg nélkül.

4.8 Tesztelés

4.8.1 Automatizált tesztek

A szerveren egy integrációs teszt (`ApplicationTest.kt`) ellenőrzi az alapvető szerver-indítást és az API gyökérvégpontját, Ktor `testApplication` segítségével.

4.8.2 Manuális tesztelés

Kiterjedt manuális tesztelés történt az összes célplatformon: Desktop (Linux Ubuntu, Windows 10/11), Web WASM (Chrome, Firefox), Web JS (Chrome), Android (fizikai eszköz) és iOS (fizikai eszköz, Xcode).

4.8.3 Azonosított és javított hibák

A fejlesztés során – különösen 2026 februárjában – számos hiba azonosítása és javítása történt. Az alábbiakban a legjelentősebbek:

1. Push értesítés architektúra teljes átdolgozása. Az első implementáció `coroutine scope` kezelése hibás volt, hiányzott a `NotificationData` osztály, és az értesítési adatok szétszórt paraméterezéssel működtek. A `FirebasePushService` teljes újraírásra került egységes adatosztállyal és megfelelő erőforrás-felszabadítással.

2. Firebase push token csak Android platformon. A `firebase-messaging` KMP könyvtár importja a `commonMain`-ben volt, holott az csak Androidon érhető el, ami fordítási hibát okozott a többi platformon. A megoldás: a `dependency` áthelyezése `androidMain`-be, és `expect/actual` implementáció minden platformra.

3. CORS konfiguráció hiánya. CORS beállítás nélkül a webes kliensek nem tudtak kommunikálni a szerverrel. A `ktor-server-cors` `dependency` és a `CORSConfig.kt` fájl hozzáadása oldotta meg a problémát.

4. Back navigáció gombkötés web platformon. A `Backspace` billentyű vissza-navigációt váltott ki, ami ütközött a szövegbeviteli mezőkkel. A

Backspace eltávolítása a navigációs billentyűköteésekből és az Alt + Bal nyíl kombinációra való áttérés javította.

5. Profilkép betöltési rendszer race condition-ok. Az első változat komplex aszinkron mintái több platformon race condition-okat okoztak: a kép időnként nem töltődött be, vagy felesleges hálózati kéréseket küldött. A betöltési logika egyszerűsítése és platform-specifikus implementációk megírása oldotta meg.

5. Összegzés

A Vezérfonal fejlesztése sikeresen zárult. A kitűzött célok teljesültek: az alkalmazás mind az öt tervezett platformon (Android, iOS, Web WASM, Web JS, Desktop) működőképes, a célzott üzenetküldés, olvasási visszajelzés, nudge funkció, emoji reakciók, admin hierarchia, valós idejű SSE és Firebase értesítések, JWT+BCrypt+2FA autentikáció, valamint a séma-szintű multi-tenant architektúra mind megvalósult.

A projekt értékes tapasztalatokkal járt. Technológiai oldalon a Kotlin Multiplatform és Compose Multiplatform valóban lehetővé teszi a többplatformos fejlesztést, de a WebAssembly és iOS integráció komoly platform-specifikus munkát igényel. Szervezési oldalon a legfontosabb tanulság az időgazdálkodás: az alaposabb előzetes tervezés és feladatbontás sok utólagos újratervezést megelőzhetett volna. A kétfős csapat jól működött – a közvetlen kommunikáció és az egyértelmű felelősségek gyors döntéshozatalt tettek lehetővé.

6. Továbbfejlesztési lehetőségek

Az alkalmazás jelenleg éles üzemeltetés felé tart: a `vezerfonal.org` domain regisztrálva van, a szerver telepítése folyamatban van. A közeli tervek között szerepel, hogy különböző szervezeteket keresünk fel visszajelzés-gyűjtés céljából, az iparági validáció érdekében.

A konkrét technikai továbbfejlesztési irányok közül a legfontosabbak: az Apple és Google OAuth bejelentkezés befejezése (a kódbázisban az OAuth konfiguráció és provider modell már megtalálható); web push értesítések és iOS APNs integráció, amellyel a valós idejű értesítések az összes platformra kiterjednének; fájl mellékletek hozzáadása az üzenetekhez (jelenleg csak szöveges üzenetek támogatottak); csoporthierarchia bevezetése a nagyobb szervezetek struktúrájának modellezéséhez; audit napló adminoknak az átláthatóság növelésére; valamint teljes szöveges keresés az üzenetekben.

7. Irodalomjegyzék

Hivatalos dokumentációk:

1. Kotlin programozási nyelv és Kotlin Multiplatform – JetBrains hivatalos dokumentáció. Elérhető: <https://kotlinlang.org>
2. Ktor szerver és kliens keretrendszer – JetBrains hivatalos dokumentáció. Elérhető: <https://ktor.io>
3. Android fejlesztői útmutatók és codelabs – Google Android dokumentáció. Elérhető: <https://developer.android.com>

Könyvtárak és keretrendszerek (GitHub):

4. Exposed ORM – JetBrains. Elérhető: <https://github.com/JetBrains/Exposed>
5. Compose Multiplatform – JetBrains. Elérhető: <https://github.com/JetBrains/compose-multiplatform>
6. Voyager Navigator – Adriel Café. Elérhető: <https://github.com/adrielcafe/voyager>
7. Firebase Admin Java SDK – Google. Elérhető: <https://github.com/firebase/firebase-admin-java>
8. gitlive Firebase KMP – GitLive. Elérhető: <https://github.com/GitLiveApp/firebase-kotlin-sdk>
9. KVault – Liftric. Elérhető: <https://github.com/Liftric/KVault>
10. Multiplatform Settings – Russell Wolf. Elérhető: <https://github.com/russhwolf/multiplatform-settings>
11. BCrypt – Favre. Elérhető: <https://github.com/patrickfav/bcrypt>
12. Resend Java SDK – Resend. Elérhető: <https://github.com/ resend/resend-java>